Arm Based ECE4813-SoC — Suit Stack Race

DE10-Nano Cyclone V FPGA / dual-core Arm Cortex-A9 | Vision-based card game

Team Members: Goh Swee Thian (33589674), Wai Wen Kang (34649395), Mandhirej Singh (34640665)

Revision: 2 Date: 27 Oct 2026 GitHub: https://github.com/mandhirej/ECE4813_Project
Video Evidence: pre-demo video

System overview

Suit Stack Race is a two-player, camera-guided playing-card game using a 4 × 4 placement grid and a fifth-column DRAW slot. A Terasic D8M camera provides the live board image. The Arm Cortex-A9 HPS performs crop/board processing, turn rules and input control while a custom FPGA CNN recognises cards and hand gestures. The FPGA also draws the 1920 × 1080 HDMI interface and timer, displaying on an external monitor.

Intended for tabletop game demonstration using a FPGA-HPS co-design approach. Cards/gestures must remain inside marked camera zones with adequate illumination; detection is not guaranteed under occlusion, shadows or movement.

Key features

• The classifier supports all 52 standard playing cards and two jokers. It also distinguishes a fist, palm, peace sign and empty hand, using separate class IDs for the four gestures.

• The game guides each turn with a live board overlay, an automatic DRAW scan and placement verification. Joker placement and removal are handled by the HPS game engine.

• Players may use keyboard commands or hand gestures: PEACE acts as ENTER, FIST as J and PALM as Q. Gesture mode requires a stable three-second hold followed by release to reduce accidental activation; PALM also uses a 75% agreement threshold.

System block diagram

Figure 1 shows how the captured camera image, CNN accelerator, HPS software and HDMI display exchange data across the FPGA–HPS bridges.

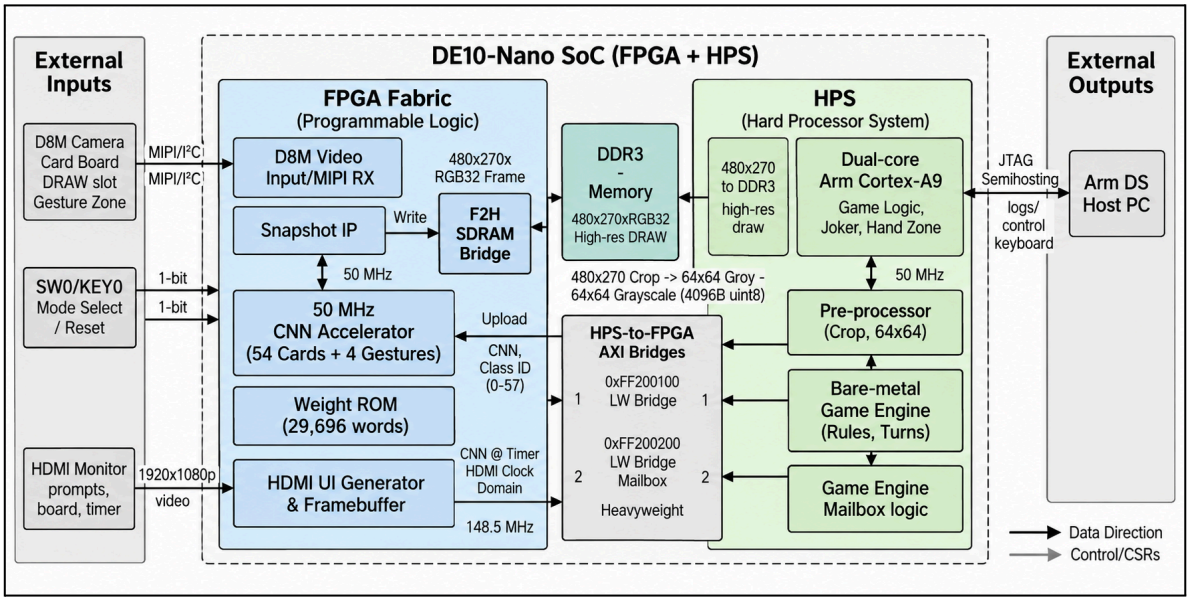


Figure 1: System block diagram

The CNN and related control logic use the 50 MHz FPGA clock, whereas HDMI output uses its dedicated pixel clock. The HPS communicates with FPGA peripherals through memory-mapped interfaces, KEY0 provides an active-low board reset.

Implementation summary

HPS: bare-metal Arm game engine: camera cropping, game rules and input control.

FPGA: video capture and snapshot, a 2-layer CNN + fully connected classifier, and the HDMI game UI.

Input: D8M camera video, plus player commands by hand gesture or Arm DS keyboard (switched with SW0 or the H key).

Output: HDMI game display and Arm DS console log.

Main limits: needs a fixed tabletop layout and steady lighting. On the board, DRAW card recognition was about 90% correct, and gestures were reliable only in the trained pose.

Implementation and interfaces

Design configuration

Item	Implemented value or configuration
Hardware and build	Terasic DE10-Nano, Cyclone V 5CSEBA6U23I7, Quartus revision DE10_Nano_D8M_DDR3
Clocks and reset	FPGA CNN/control 50 MHz (FPGA_CLK1_50), HDMI 148.5 MHz PLL output, HPS MPU 800 MHz, DDR3 400 MHz (preloader PLL configuration, configured). KEY0 active-low reset.
Memory and data	HPS DDR3: 480 × 270 RGB32 frame (518,400 B), plus high-resolution DRAW window, CNN 64 × 64 grayscale uint8 (4096 B). FC weight ROM: 29,696 words.
Software and tools	Bare-metal C, Arm Development Studio 2026.0-1 / arm-eabi-gcc (Linaro GCC 7.5-2019.12), Quartus Prime 25.1std.0 Lite (Build 1129), Platform Designer system, Verilog RTL, Python + Icarus Verilog test utilities.
Algorithm or processing	3 × 3 Conv1 (8 filters, stride 2) → 3 × 3 Conv2 (16 filters, stride 2) → int8 / fixed-point FC argmax. Cards 0–53, gestures 54–57.

Interface map

Directions relative to the DE10-Nano, D8M/HDMI use built-in connectors (no GPIO jumpers).

Signal or port	Owner / direction	Connector, pin assignment or setting
D8M image / I²C	FPGA input / I²C	D8M adapter, MIPI_PIXEL_CLK=PIN_E8, D[9:0] + HS(PIN_Y19) / VS(PIN_AC23).
HDMI video	FPGA output	On-board HDMI TX, CLK=PIN_AG5, RGB[23:0] + DE/HS/VS, 1920 × 1080.
SW0, KEY0	FPGA input	SW[0]=PIN_Y24 (input-mode select), KEY[0]=PIN_AH17 (active-low reset).
Arm DS console / JTAG	HPS bidirectional	Host PC via board USB-Blaster/JTAG, semihosting text input/output, no UART baud setting.
Snapshot / CNN CSRs	HPS bi-dir FPGA	LW bridge: snapshot 0xFF200100, CNN upload 0xFF200200, status 0xFF200020.
HDMI game mailbox	HPS → FPGA	LW bridge 0xFF200000 (status) / +0x10 (command), shared resets/handshake.

Processing and software flow

D8M FPGA snapshot → HPS DDR → board/gesture crop → 64 × 64 grayscale image → 4096-byte CNN upload → FPGA class ID → HPS game state → HDMI UI. DRAW uses an optional high-resolution window (fallback to 480 × 270). Gesture/card FC class ranges are 54–57 / 0–53, HPS checks bridge ID, hash, status, timeouts and class validity.

Build and operation

1. Connect D8M and HDMI, align the 4 × 4 board, DRAW and HG zones under the camera.
2. Open DE10_Nano_D8M_DDR3.qpf (Quartus 25.1 Lite), compile, then program output_files/DE10_Nano_D8M_DDR3_time_limited.sof (time-limited build: keep the USB-Blaster connected). Do not regenerate Platform Designer.
3. Arm DS: import Atlas-Blinking-LED-Baremetal-GNU, Clean/Build and launch u-boot-spl.axf → atlas_main.axf.
4. Console: G game, H/SW0 gesture mode, C capture, Q stop. Hold FIST/PEACE/PALM for 3 s in HG and follow HDMI.

GitHub: https://github.com/mandhirej/ECE4813_Project. Core files: DE10_Nano_D8M_DDR3_CNN/{*.qpf, *.qsf, CNN.v, soc_system/, capture_ip/, weights_linear_gesture.mif}, Atlas-Blinking-LED-Baremetal-GNU/{atlas_main.c, *.h, u-boot-spl.axf}, CNN_train_model/{card_model/, gesture_model/, simulation/}.

Performance and validation

Evidence below separates software/RTL simulation from physical-board verification, no hardware latency or current measurements are claimed.

Functional performance table

Verified evidence from repository documentation, scripts, exported model report and Quartus build outputs, plus manual observation on the board:

Input or test case	Expected output	Observed result	Evidence / scope
54-class card CNN + gestures	FPGA class matches integer model	40 / 40 input images, RTL match, original card classes unchanged in compared cases	simulation/run_sim.sh, RTL simulation

Input or test case	Expected output	Observed result	Evidence / scope
HPS gesture crop	Bit-exact trained preprocessing	300 / 300 zone samples match Python	gesture_model/check_hps_preprocess.py
Held-out gesture photos	Fist, palm, peace, empty	25 / 26 (96.2%), cross-session 88.2% / 86.2%	gesture_model/export/report.txt, offline
Full game + DRAW	16 legal placements and joker actions	16 / 16 cards handled in a 329 s virtual game, high-res snapshot words checked under stalls	Manual observation during board play on HDMI display and ARM terminal log
Physical D8M hand control	Reliable 3 s hold on live camera	Reliable, no misfires observed, when held in the trained pose (fingers up, arm entering HG from the bottom)	Manual observation during board play on HDMI display
Card in DRAW (board)	Correct card recognised and target shown	About 90% correct (informal count during play, not a counted trial set)	Manual observation during board play
Placement check + joker detection (board)	Correct placement verified, opponent joker found	Passed in all observed turns	Manual observation during board play on HDMI display
Incorrect arrangement of card	Incorrect card placement flagged, request for change	Always flags incorrect placement and request for change consistently	Manual observation during board play on HDMI display

Response and other available results

CNN inference (FPGA only, simulated): 425,116 cycles ≈ 8.5 ms per gesture; 732,366 cycles ≈ 14.6 ms per card, at 50 MHz. These are RTL simulation cycle counts, not camera-to-screen latency.

Observed response (manual, approximate): about 1–2 s from the card being placed to the result appearing on HDMI. A gesture command additionally needs a 3 s steady hold, then the hand removed.

Quartus results (23 Oct 2026 build, tool-reported): 19,910 / 41,910 ALMs (48%); RAM blocks 495 / 553 (90%); memory bits 68%; DSP 9 / 112 (8%). The fit succeeded, but timing did not close: worst setup slack −17.125 ns on FPGA_CLK1_50 (slow −40 °C model), with 18 of 192 timing checks failing. These figures describe the hardware build, not game performance.

Applications and limits

Applications: The design demonstrates FPGA-accelerated image recognition and HPS-controlled game logic in an interactive tabletop application. It can also be used as an educational example of hardware/software partitioning and contact-free input.
Limits: Recognition is sensitive to lighting, card orientation and hand pose; palm and peace gestures may be confused. The FPGA RAM is nearly full, setup timing remains negative, and the live detection estimates are observational rather than statistically measured.

Test circuit and setup

Physical arrangement and signal routing (camera adapter and HDMI use the board's dedicated connectors):

4 × 4 board + DR / HG D8M camera via board adapter ↓ MIPI + I²C to FPGA	DE10-Nano FPGA/HPS Snapshot → DDR → CNN/class ID FPGA HDMI framebuffer / UI	HDMI monitor ← HDMI_TX_CLK + RGB[23:0], DE, HS, VS 1080p video output
SW0: PIN_Y24 → gesture mode KEY0: PIN_AH17 → reset	USB-Blaster/JTAG to Arm DS PC HPS semihosting + programming	Common ground via the module/HDMI/board connectors, no user-wired GPIO jumpers

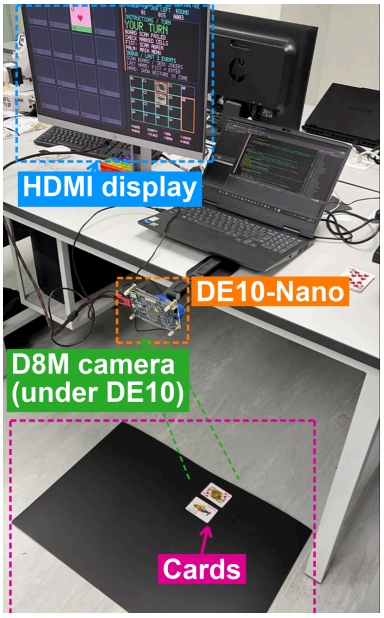


Figure 2: Test Setup